

תרגיל מספר 3- משחק ללימוד HTTP ו-REST

תיאור המטלה

בתרגיל זה עליכם לבנות משחק אינטרנטי קצר שמטרתו לתרגל ולהמחיש את אופן התקשורת בין צד הלקוח לצד השרת באמצעות HTTP.

המשחק יהיה בנוי ממספר שלבים. בכל שלב יוצג למשתמש תרחיש המתאר פעולה שמתרחשת במערכת אינטרנטית, ועל המשתמש להרכיב את הבקשה המתאימה לביצוע הפעולה: לבחור את סוג הבקשה, את הנתיב ואת הפרמטרים המתאימים, ולשלוח את הבקשה לשרת.

לדוגמה: "המשתמש רוצה להציג רק מוצרים מקטגוריית Books, הממוינים לפי מחיר."

הבקשות במשחק אינן סימולציה. על המשחק לשלוח בפועל בקשות HTTP לשרת שבניתם, לקבל ממנו תשובות אמיתיות ולהציג למשתמש את התוצאה.

דרישות מינימום

על המשחק להכיל את כל הרכיבים הבאים:

- לפחות 8 שלבים שונים.
- בכל שלב יוצג תיאור ברור של הפעולה שהמשתמש נדרש לבצע.
- בכל שלב יוצגו למשתמש אפשרות לבחור HTTP Method, אפשרות להגדיר את נתיב הבקשה, ובמידת הצורך אפשרות להזין Route Parameters, Query Parameters או Request Body - וכן כפתור לשליחת הבקשה.
- לאחר שליחת הבקשה תוצג למשתמש תשובת השרת, לרבות ה-Status Code והמידע שהוחזר.
- בדיקת נכונות הפתרון תתבצע בצד השרת: יקבע האם הבקשה שהתקבלה מתאימה לדרישות השלב, ויחזיר לצד הלקוח תשובה בהתאם. אין לממש את בדיקת הנכונות בצד הלקוח.
- בכל בקשה במסגרת המשחק יישלח לשרת גם מזהה השלב הנוכחי. השרת ישתמש במזהה זה כדי לבדוק האם ה-Method, הנתיב, הפרמטרים וה-Body של הבקשה תואמים לדרישות השלב. הגדרת הפתרונות הנכונים לכל שלב תישמר בצד השרת בלבד ולא תופיע בקוד צד הלקוח.
- במקרה של פתרון נכון תוצג הודעת הצלחה ותתאפשר התקדמות לשלב הבא.
- במקרה של פתרון שגוי תוצג הודעה מתאימה והמשתמש יוכל לשנות את הבקשה ולנסות שוב.
- יש להציג למשתמש באיזה שלב הוא נמצא, לדוגמה: שלב 4 מתוך 8.
- המעבר בין שלבי המשחק יתבצע ללא מעבר לעמוד HTML חדש.

נושאים שחייבים להופיע בשלבי המשחק

השלבים צריכים לתרגל מגוון מצבים ולא לחזור על אותה פעולה עם ערכים שונים בלבד. במהלך המשחק יש לעשות שימוש לפחות בנושאים הבאים:

- בקשות GET.
- בקשות POST.
- בקשת PUT או PATCH.
- בקשת DELETE.
- שימוש ב-Route Parameters.
- שימוש ב-Query Parameters, כולל שלב אחד לפחות הדורש יותר מ-Query Parameter אחד. על ה-Query Parameters להשפיע בפועל על הנתונים שהשרת מחזיר, לדוגמה לצורך סינון, חיפוש או מיון.
- שימוש ב-Request Body.
- החזרת JSON מהשרת.
- שימוש ב-HTTP Status Codes מתאימים.
- לפחות שלב אחד שבו השרת מחזיר שגיאה בעקבות בקשה שאינה תקינה (לדוגמה: בקשה למשאב שאינו קיים), והמשתמש נדרש לבחון את תגובת השרת.

לפחות שלושה מהשלבים צריכים לדרוש שילוב של יותר ממושג אחד. לדוגמה: שלב הדורש גם Route Parameter וגם Request Body, או שילוב של מספר Query Parameters באותה בקשה.

תכנון השלבים

עליכם לתכנן בעצמכם את שלבי המשחק ואת מבנה הנתונים שעליו המשחק מתבסס. כל שלב צריך לבדוק הבנה שונה או שילוב שונה של מושגים - אין לבנות מספר שלבים כמעט זהים שההבדל ביניהם הוא רק שינוי של מספר מזהה, שם מוצר או ערך אחר.

לדוגמה, השלבים "הצג מוצר 1", "הצג מוצר 2", "הצג מוצר 3" ייחשבו לאותו סוג של שלב. לעומת זאת, רצף כגון: "הצג מוצר מסוים", "הצג מוצרים מקטגוריה מסוימת", "הוסף מוצר", "עדכן מוצר", "מחק מוצר", "הצג מוצרים לפי מספר תנאים" - הוא רצף תקין, שבו כל שלב דורש מהמשתמש להבין פעולה שונה.

בניית השרת

- עליכם לבנות שרת באמצעות Node.js ו-Express.
- השרת יכיל לפחות שני משאבים שונים לבחירתכם, לדוגמה: מוצרים וביקורות, ספרים ומחברים, סרטים ובמאים, אירועים ומשתתפים. אין חובה להשתמש בדוגמאות אלו.
- מומלץ ששלב אחד לפחות במשחק ינצל את הקשר בין המשאבים, לדוגמה: הצגת כל הביקורות של מוצר מסוים.
- יש לממש עבור המשאבים את ה-Routes המתאימים לכל הפעולות שהמשחק דורש.
- הנתונים יישמרו בצד השרת בלבד - ייטענו מקובץ JSON או יוגדרו מראש בקוד. אין להשתמש במסד נתונים.
- פעולות של הוספה, עדכון ומחיקה ישנו בפועל את הנתונים בזיכרון השרת, כך שהמשתמש יוכל לראות את השינוי בבקשות המשך (אין צורך לשמור את השינויים לאחר הפעלה מחדש של השרת).

REST, AJAX ו-SSR

- על ה-API שתבנו להיות מבוסס על עקרונות REST: בחירת HTTP Methods ונתיבים המתאימים לפעולות, ומבנה Routes עקבי וברור. לדוגמה, אין ליצור Route בשם `/deleteBook` אם ניתן לייצג את הפעולה באמצעות HTTP Method ונתיב מתאים למשאב.
- שליחת הבקשות במשחק תתבצע באמצעות AJAX, ללא טעינה מחדש של העמוד. לאחר קבלת התשובה יש לעדכן את ממשק המשתמש בהתאם: הצגת ה-Status Code, הנתונים שהוחזרו והודעת הצלחה או שגיאה.
- עמוד המשחק הראשי ייווצר בצד השרת באמצעות EJS.
- בנוסף, יש ליצור עמוד סכמות: עמוד SSR נוסף המציג את הסכמה של כל אחד מהמשאבים במערכת - שם המשאב, השדות שלו וסוג הנתונים של כל שדה. נתוני העמוד יישלחו מהשרת ויכללו בתבנית ה-EJS בזמן יצירת העמוד.
- כל נתיבי ה-API ירוכזו תחת `/api`. עמוד המשחק יהיה ב-`/` ועמוד הסכמות ב-`/schemas`.

בנוסף, על האתר לעמוד בדרישות הבאות:

- שימוש ב-Vanilla JavaScript בלבד בצד הלקוח.
- שימוש נכון ב-HTTP Methods וב-HTTP Status Codes.
- הפרדה ברורה בין צד הלקוח לצד השרת.
- האתר יהיה רספונסיבי ויעבוד בצורה תקינה גם במסכי מובייל וגם במסכים גדולים.
- קוד ה-CSS וקוד ה-JavaScript של צד הלקוח יופרדו לקבצים חיצוניים.

הערה חשובה

נושא המשחק והמשאבים שבו לבחירתכם, כמו בתרגיל הקודם - חנות, ספרייה, אוסף סרטים או כל רעיון אחר.

הדגש בתרגיל הוא על הבנה של תקשורת Client-Server, על שימוש נכון ב-HTTP, REST, Route, Parameters, Query Parameters, AJAX ו-SSR - ולא על מורכבות העיצוב הגרפי.

הנחיות הגשה:

1. ההגשה מתבצעת בזוגות בלבד. על כל אחד מבני הזוג להגיש את המטלה במודל ולהוסיף בהערה להגשה את פרטי בן/בת הזוג. הגשה ביחידים או בכל הרכב אחר לא תיבדק וציונה יהיה 0.
2. יש להעלות את הפרויקט ל-GitHub. שימו לב: בתרגיל זה אין להשתמש ב-GitHub Pages, מכיוון שמדובר באפליקציה עם צד שרת שאינה יכולה לרוץ שם.
3. בתיבת ההגשה במודל יש לצרף: קישור ל-Repository וקובץ ZIP המכיל את קבצי הפרויקט (ללא תיקיית node_modules).
4. הפרויקט יכלול קובץ README עם הוראות התקנה והרצה מלאות (npm install, פקודת ההרצה והכתובת בדפדפן). פרויקט שלא ניתן להריץ לפי ההוראות ייחשב כאי-הגשה.
5. ודאו שה-Repository ציבורי (Public).
6. לא ניתן להגיש לאחר התאריך המופיע במודל כלל.

דרישות המינימום המפורטות לעיל הן תנאי סף בלבד. לציון גבוה יש להגיש תרגיל מלא, שלם ומתפקד (מעשית ולוגית), המכיל מעבר לדרישות המינימום. לדוגמה:

- שלבים נוספים ומגוונים המשלבים מספר מושגים בכל שלב.
- ממשק ברור ונוח להרכבת הבקשות, עם עיצוב נקי ועקבי.
- טיפול מוקפד במקרי שגיאה, בצד הלקוח ובצד השרת.
- משוב ויזואלי עשיר למשתמש: הבחנה ברורה בין הצלחה לשגיאה, הצגה מסודרת של תשובת השרת.
- מערכת ניקוד, ספירת ניסיונות או שמירת התקדמות המשתמש במהלך המשחק.
- אפשרות לחזור לשלבים שכבר הושלמו.

בהצלחה!